

课程编码	适用产品	产品版本	课程版本

作者/工号	时间	审核人/工号	新开发/优化
李春华/00891885	2024.07.22	新开发	

C++企业软件开发实践

实验3-1 面向对象编程



前言

欢迎参加C++企业软件开发实践课程。本课程旨在通过完整开发案例，分享企业软件开发中的实践、经验和要求，帮助在校学生提升软件开发技能，养成良好的软件开发习惯。

实践课程共有4次实验，本课程为实验3，您将继续实践面向对象编程，包括：

- **表驱动**：掌握面向对象多态特性融合C++ STL库的使用，进一步提升代码的扩展性
- **分层设计与抽象**：掌握有效解耦代码中的循环依赖，从而提高代码的可维护性和可扩展性

期待您在课程中的精彩表现！

目标

通过本课程的学习，您将能够：

- 表驱动和STL容器、智能指针
 - 掌握C++ STL库的融合使用：通过学习如何有效地使用STL容器和智能指针，提升代码的可扩展性和灵活性
 - 提高代码的可扩展性：通过表驱动的设计模式，增强代码的可维护性和可读性
- 解耦循环依赖
 - 分层设计和抽象化：学习如何通过分层设计和抽象化来有效地解耦代码中的循环依赖
 - 提高代码的可维护性和可扩展性：通过解耦循环依赖，提升代码的可维护性和可扩展性，使代码更易于理解和修改

这些学习目标旨在帮助您更好地掌握C++编程的最佳实践，建立高质量编程的**意识**，养成良好的开发**习惯**，提高代码质量和开发效率。

目录

1. 实验2回顾
2. 项目实践
 - 2.1 数据抽离解耦循环依赖
 - 2.2 表驱动提升扩展性
 - 2.3 状态抽象提升可读性
3. 总结

实验2回顾

```
void ExecutorImpl::Execute(const std::string& commands) noexcept
{
    for (const auto cmd : commands) {
        std::unique_ptr<ICommand> cmdr;

        if (cmd == 'M') {
            cmdr = std::make_unique<MoveCommand>();
        } else if (cmd == 'L') {
            cmdr = std::make_unique<TurnLeftCommand>();
        } else if (cmd == 'R') {
            cmdr = std::make_unique<TurnRightCommand>();
        } else if (cmd == 'F') {
            cmdr = std::make_unique<FastCommand>();
        }

        if (cmdr) {
            cmdr->DoOperate(*this);
        }
    }
}
```

- 通过封装、父类抽象，完成了指令处理结构的建立
- F指令的处理，依然需要修改指令处理主干逻辑

目录

- 1. 实验2回顾
- 2. 项目实践
 - 2.1 数据抽离解耦循环依赖
 - 2.2 表驱动提升扩展性
 - 2.3 状态抽象提升可读性
- 3. 总结

面向对象编程：指令处理的类独立到文件中

创建src/Command.hpp

```
#pragma once
#include "ExecutorImpl.hpp"

namespace adas
{
class ICommand
{
public:
    virtual ~ICommand() = default;
    virtual void DoOperate(ExecutorImpl& executor) const noexcept = 0;
};

class MoveCommand final : public ICommand
{
public:
    void DoOperate(ExecutorImpl& executor) const noexcept override
    {
        ...
    }
};
```

```
class TurnLeftCommand final : public ICommand
{
public:
    void DoOperate(ExecutorImpl& executor) const noexcept override
    {
        ...
    }
};

class TurnRightCommand final : public ICommand
{
public:
    void DoOperate(ExecutorImpl& executor) const noexcept override
    {
        ...
    }
};

class FastCommand final : public ICommand
{
public:
    void DoOperate(ExecutorImpl& executor) const noexcept override
    {
        ...
    }
};
```


面向对象编程：修改依赖方法访问权限

ExecutorImpl.hpp

```
#pragma once
#include "Executor.hpp"

namespace adas
{
class ExecutorImpl final : public Executor
{
public:
    void Move(void) noexcept;
    void TurnLeft(void) noexcept;
    void TurnRight(void) noexcept;
    void Fast(void) noexcept;
    bool IsFast(void) const noexcept;
private:
    Pose pose;
    bool fast{false};
};
} // namespace adas
```

ExecutorImpl.cpp

```
...
#include "Command.hpp"
...
```

编译运行验证后，代码及时入库

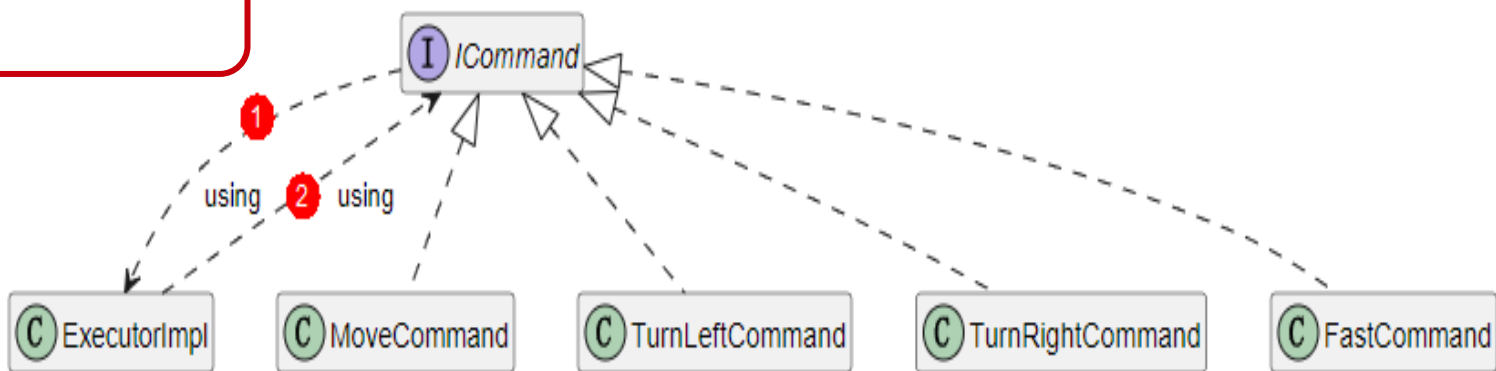
面向对象编程：代码问题分析

```
void ExecutorImpl::Execute(const std::string& commands)
noexcept
{
    for (const auto cmd : commands) {
        std::unique_ptr<ICommand> cmdr;

        if (cmd == 'M') {
            cmdr = std::make_unique<MoveCommand>();
        } else if (cmd == 'L') {
            cmdr = std::make_unique<TurnLeftCommand>();
        } else if (cmd == 'R') {
            cmdr = std::make_unique<TurnRightCommand>();
        } else if (cmd == 'F') {
            cmdr = std::make_unique<FastCommand>();
        }

        if (cmdr) {
            cmdr->DoOperate(*this);
        }
    }
}
```

```
class ICommand
{
public:
    virtual ~ICommand() = default;
    virtual void DoOperate(ExecutorImpl& executor) const
noexcept = 0;
};
```



Command <-> ExecutorImpl 循环依赖，该如何解？

面向对象编程：循环依赖的根源分析

```
class TurnLeftCommand final : public ICommand
{
public:
    void DoOperate(ExecutorImpl& executor) const noexcept override
    {
        if (executor.IsFast()) {
            executor.Move();
        }

        executor.TurnLeft();
    }
};
```

Command依赖的是
ExecutorImpl的车状态数据

```
class TurnRightCommand final : public ICommand
{
public:
    void DoOperate(ExecutorImpl& executor) const noexcept override
    {
        if (executor.IsFast()) {
            executor.Move();
        }

        executor.TurnRight();
    }
};
```

```
void ExecutorImpl::Move() noexcept
{
    if (pose.heading == 'E') {
        ++pose.x;
    }
    ...
}
```

```
void ExecutorImpl::TurnLeft() noexcept
{
    if (pose.heading == 'E') {
        pose.heading = 'N';
    } else if (pose.heading == 'N') {
        ...
    }
}
```

```
void ExecutorImpl::TurnRight() noexcept
{
    if (pose.heading == 'E') {
        pose.heading = 'S';
    } else if (pose.heading == 'S') {
        ...
    }
}
```

```
bool ExecutorImpl::IsFast() const noexcept
{
    return fast;
}
```

面向对象编程：解耦循环依赖，封装抽离ExecutorImpl状态数据

创建src/PoseHandler.hpp

```
#pragma once
#include "Executor.hpp"
namespace adas
{
class PoseHandler final
{
public:
    PoseHandler(const Pose& pose) noexcept;
    PoseHandler(const PoseHandler&) = delete;
    PoseHandler& operator=(const PoseHandler&) = delete;

public:
    void Move(void) noexcept;
    void TurnLeft(void) noexcept;
    void TurnRight(void) noexcept;
    void Fast(void) noexcept;
    bool IsFast(void) const noexcept;
    Pose Query(void) const noexcept;

private:
    Pose pose;
    bool fast{false};
};
} // namespace adas
```

创建src/PoseHandler.cpp

```
#include "PoseHandler.hpp"

namespace adas
{
PoseHandler::PoseHandler(const Pose& pose) noexcept :
    pose(pose)
{
}

void PoseHandler::Move() noexcept
{
    if (pose.heading == 'E') {
        ++pose.x;
    } else if (pose.heading == 'W') {
        --pose.x;
    } else if (pose.heading == 'N') {
        ++pose.y;
    } else if (pose.heading == 'S') {
        --pose.y;
    }
}
```

面向对象编程：解耦循环依赖，封装抽离ExecutorImpl状态数据

src/PoseHandler.cpp

```
void PoseHandler::TurnLeft() noexcept
{
    if (pose.heading == 'E') {
        pose.heading = 'N';
    } else if (pose.heading == 'N') {
        pose.heading = 'W';
    } else if (pose.heading == 'W') {
        pose.heading = 'S';
    } else if (pose.heading == 'S') {
        pose.heading = 'E';
    }
}
```

```
void PoseHandler::TurnRight() noexcept
{
    if (pose.heading == 'E') {
        pose.heading = 'S';
    } else if (pose.heading == 'S') {
        pose.heading = 'W';
    } else if (pose.heading == 'W') {
        pose.heading = 'N';
    } else if (pose.heading == 'N') {
        pose.heading = 'E';
    }
}
```

面向对象编程：解耦循环依赖，封装抽离ExecutorImpl状态数据

src/PoseHandler.cpp

```
Pose PoseHandler::Query() const noexcept
{
    return pose;
}

void PoseHandler::Fast() noexcept
{
    fast = !fast;
}

bool PoseHandler::IsFast() const noexcept
{
    return fast;
}
} // namespace adas
```

面向对象编程：解耦循环依赖，封装抽离ExecutorImpl状态数据

src/Command.hpp

```
#include "PoseHandler.hpp"
...
class ICommand
{
public:
    virtual ~ICommand() = default;
    virtual void DoOperate(PoseHandler& poseHandler) const noexcept = 0;
};

class MoveCommand final : public ICommand
{
public:
    void DoOperate(PoseHandler& poseHandler) const noexcept override
    {
        if (poseHandler.IsFast()) {
            poseHandler.Move();
        }

        poseHandler.Move();
    }
};
```

```
class TurnLeftCommand final : public ICommand
{
public:
    void DoOperate(PoseHandler& poseHandler) const noexcept override
    {
        if (poseHandler.IsFast()) {
            poseHandler.Move();
        }

        poseHandler.TurnLeft();
    }
};

class TurnRightCommand final : public ICommand
{
public:
    void DoOperate(PoseHandler& poseHandler) const noexcept override
    {
        if (poseHandler.IsFast()) {
            poseHandler.Move();
        }

        poseHandler.TurnRight();
    }
};
```


面向对象编程：解耦循环依赖，封装抽离ExecutorImpl状态数据

src/Command.hpp

```
class FastCommand final : public ICommand
{
public:
    void DoOperate(PoseHandler& poseHandler) const noexcept override
    {
        poseHandler.Fast();
    }
};
```

面向对象编程：解耦循环依赖，封装抽离ExecutorImpl状态数据

src/ExecutorImpl.hpp

```
...  
#include "PoseHandler.hpp"  
...  
class ExecutorImpl final : public Executor  
{  
public:  
...  
private:  
    PoseHandler poseHandler;  
};
```

编译运行验证后，代码及时入库

src/ExecutorImpl.cpp

```
ExecutorImpl::ExecutorImpl(const Pose& pose) noexcept :  
    poseHandler(pose)  
{  
}  
  
void ExecutorImpl::Execute(const std::string& commands) noexcept  
{  
    ...  
    for (const auto cmd : commands) {  
        const auto it = cmdMap.find(cmd);  
  
        if (it != cmdMap.end()) {  
            it->second->DoOperate(poseHandler);  
        }  
    }  
}  
  
Pose ExecutorImpl::Query() const noexcept  
{  
    return poseHandler.Query();  
}
```

本节小结

通过数据抽离和代码分层策略提高代码可扩展性和可维护性：

- 抽离数据：
 - 演示如何通过抽离数据来解耦代码中的循环依赖
 - 强调数据抽离在简化代码结构和减少依赖关系中的作用
- 代码分层策略：
 - 介绍代码分层策略及其在解决相互依赖问题中的应用
 - 通过具体示例展示如何分层代码以实现更清晰的结构和更低的耦合度
- 封装和抽离Executor的状态数据：
 - 解释如何通过封装和抽离Executor的状态数据来消除循环依赖
 - 展示这种方法如何提高代码的可扩展性和可维护性
- 提升代码质量：
 - 强调通过数据抽离和代码分层策略，可以显著提升代码的可扩展性和可维护性

目录

- 1. 实验2回顾
- 2. 项目实践
 - 2.1 数据抽离解耦循环依赖
 - 2.2 表驱动提升扩展性
 - 2.3 状态抽象提升可读性
- 3. 总结

课程实训需求2-2 支持倒车指令

Executor组件增加执行倒车指令：

B: 倒车指令，接收到该指令，车进入倒车状态，该状态下：

- M：在当前朝向上后退一格，朝向不变。注：比如朝向为N时收到M指令，y坐标减1，朝向保持N
- L：右转90度，位置不变
- R：左转90度，位置不变

B和F两个状态可以叠加，叠加状态下：

- M：倒退2格（不能跳跃，只能一格一格后退）
- L：先倒退一格，然后右转90度
- R：先倒退一格，然后左转90度

再接收一次B指令，对应的状态取消

面向对象编程：B指令对指令处理结构的扩展性诉求分析

```
void ExecutorImpl::Execute(const std::string& commands) noexcept
{
    for (const auto cmd : commands) {
        std::unique_ptr<ICommand> cmdr;

        if (cmd == 'M') {
            cmdr = std::make_unique<MoveCommand>();
        } else if (cmd == 'L') {
            cmdr = std::make_unique<TurnLeftCommand>();
        } else if (cmd == 'R') {
            cmdr = std::make_unique<TurnRightCommand>();
        } else if (cmd == 'F') {
            cmdr = std::make_unique<FastCommand>();
        } else if (cmd == 'B') {
            cmdr = std::make_unique<ReverseCommand>();
        }

        if (cmdr) {
            cmdr->DoOperate(poseHandler);
        }
    }
}
```

- 通过封装、多态特性的应用，完成了指令处理结构的建立
- B指令的处理，依然需要修改指令处理主干逻辑

面向对象编程：表驱动简介

- 在C++中，可以使用std::map来存储键值对，其中键是条件，值是对应的处理函数或者对象
- 表驱动的优势：
 - 简化代码逻辑：避免了大量的条件语句，使代码更易读
 - 易于扩展：添加新的条件或和处理函数时，只需要更新表格，而不需要修改现有的逻辑
 - 提高可维护性：逻辑和数据分离，便于维护和调试

面向对象编程：应用表驱动消减圈复杂度，功能代码实现

```
#include <unordered_map>
...
void ExecutorImpl::Execute(const std::string& commands) noexcept
{
    std::unordered_map<char, std::unique_ptr<ICommand>> cmdMap;
    cmdMap.emplace('M', std::make_unique<MoveCommand>());
    cmdMap.emplace('L', std::make_unique<TurnLeftCommand>());
    cmdMap.emplace('R', std::make_unique<TurnRightCommand>());
    cmdMap.emplace('F', std::make_unique<FastCommand>());

    for (const auto cmd : commands) {
        const auto it = cmdMap.find(cmd);

        if (it != cmdMap.end()) {
            it->second->DoOperate(*this);
        }
    }
}
```

编译运行验证后，代码及时入库

本节小结

通过面向对象编程和STL库提升代码可扩展性：

- 表驱动消减圈复杂度：
 - 通过表驱动方法减少代码中的复杂循环，提高代码的可读性和维护性
- 多态特性与STL库的融合：
 - 将多态特性与STL（标准模板库）结合使用，提升代码的灵活性和可扩展性

目录

- 1. 实验2回顾
- 2. 项目实践
 - 2.1 数据抽离解耦循环依赖
 - 2.2 表驱动提升扩展性
 - 2.3 状态抽象提升可读性
- 3. 总结

面向对象编程：PoseHandler待优化代码分析

```
void PoseHandler::Move() noexcept
{
    if (pose.heading == 'E') {
        ++pose.x;
    } else if (pose.heading == 'W') {
        --pose.x;
    } else if (pose.heading == 'N') {
        ++pose.y;
    } else if (pose.heading == 'S') {
        --pose.y;
    }
}
```

(1, 0)

(-1, 0)

(0, 1)

(0, -1)

```
class Point final
{
public:
    Point(const int x, const int y) noexcept;

    Point(const Point& rhs) noexcept;
    Point& operator=(const Point& rhs) noexcept;

    Point& operator+=(const Point& rhs) noexcept;

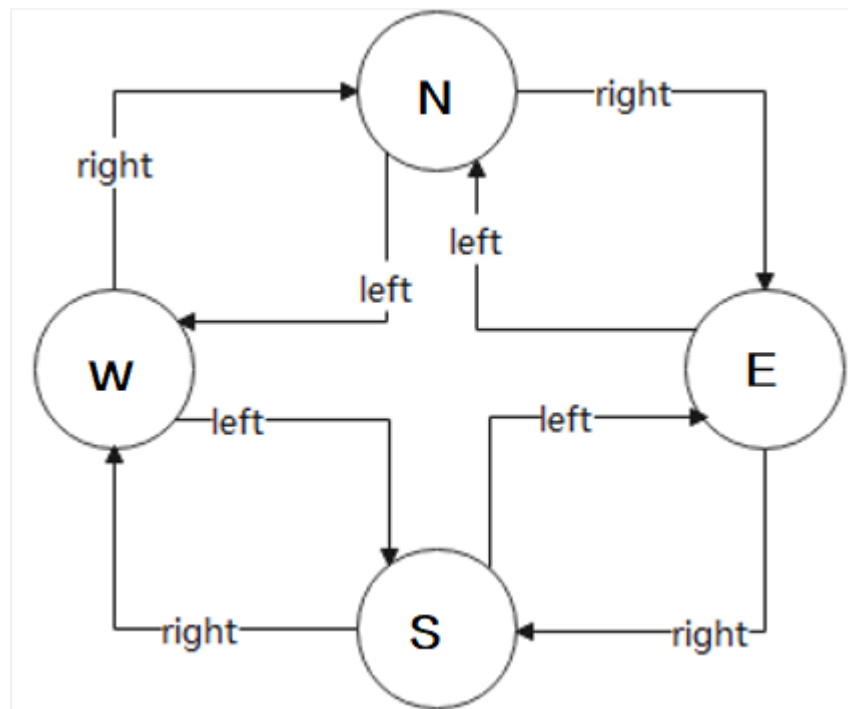
public:
    int GetX(void) const noexcept;
    int GetY(void) const noexcept;

private:
    int x;
    int y;
};
```

面向对象编程： PoseHandler待优化代码分析

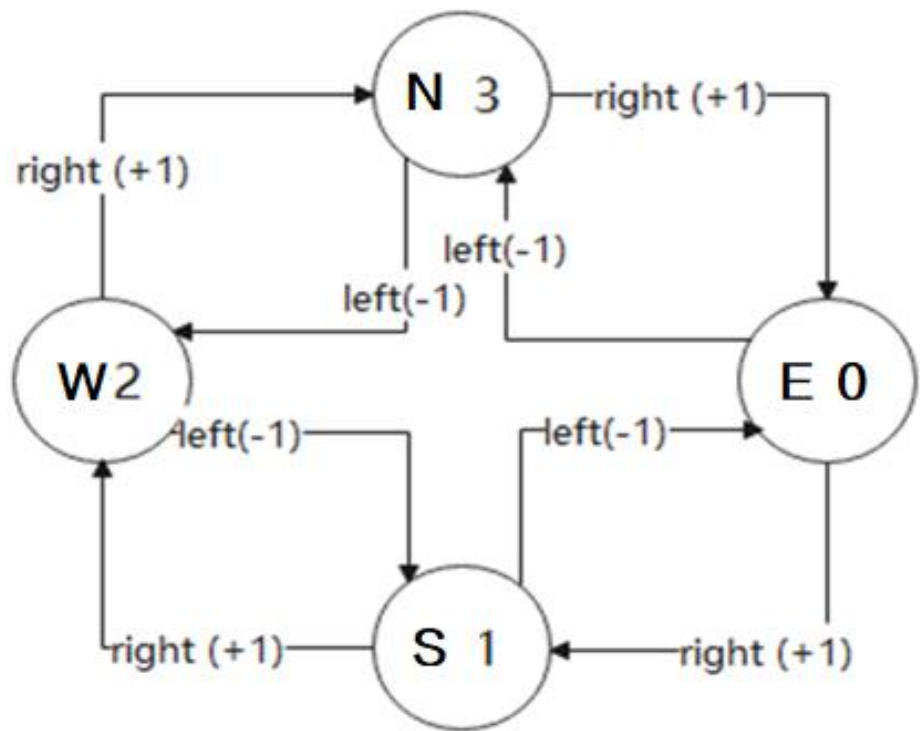
```
void PoseHandler::TurnLeft() noexcept
{
    if (pose.heading == 'E') {
        pose.heading = 'N';
    } else if (pose.heading == 'N') {
        pose.heading = 'W';
    } else if (pose.heading == 'W') {
        pose.heading = 'S';
    } else if (pose.heading == 'S') {
        pose.heading = 'E';
    }
}
```

```
void PoseHandler::TurnRight() noexcept
{
    if (pose.heading == 'E') {
        pose.heading = 'S';
    } else if (pose.heading == 'S') {
        pose.heading = 'W';
    } else if (pose.heading == 'W') {
        pose.heading = 'N';
    } else if (pose.heading == 'N') {
        pose.heading = 'E';
    }
}
```



面向对象编程：状态问题的抽象

用计算机的原始计算属性解决复杂状态流转问题



右转后状态编码 = (当前状态编码 + 1) % 4

左转后状态编码 = (当前状态编码 - 1 + 4) % 4 = (当前状态 + 3) % 4

本节小结

通过状态抽象和计算属性解决复杂状态流转问题：

- 状态问题的抽象：
 - 介绍如何将复杂的状态问题进行抽象，以简化状态管理
- 利用计算机的原始计算属性：
 - 通过计算机的基本计算属性来处理复杂的状态流转
 - 展示如何将复杂的状态转换问题简化为基本的数学运算
- 小车状态的抽象：解释如何通过这种抽象方法来简化状态管理和转换
- 指令的数字化抽象：将所有指令抽象为数字，通过加减运算来处理指令

目录

1. 实验2回顾
2. 项目实践
 - 2.1 数据抽离解耦循环依赖
 - 2.2 表驱动提升扩展性
 - 2.3 状态抽象提升可读性
3. 总结

本章总结

通过本课程的学习：

- 提升代码可扩展性：利用**表驱动**方法减少复杂循环，结合多态特性提升代码灵活性和可扩展性
- 解耦和分层策略：通过**数据抽离和代码分层**策略，解耦循环依赖，显著提升代码的可扩展性和可维护性
- 状态抽象和计算属性：通过**状态抽象**，简化复杂状态流转问题，将指令数字化处理，提高代码的可维护性和可扩展性
- 测试用例和降维分析：解析多维**降维**分析方法，设计测试防护网，确保代码的可靠性和稳定性

这些技能和知识将为您在未来的企业软件开发中打下坚实的基础，帮助您成为一名更加专业和高效的软件开发者。

感谢您参与本课程，期待您的在未来的软件开发工作中取得更大的成就！

学习推荐

在线参考资料网站，涵盖了C++基本概念到高级特性、标准库函数、类和模板等各个方面：

- 搜索功能：有强大的搜索功能，可以快速找到需要的函数、类或概念
- 示例代码：可以帮助理解如何使用C++特性
- 网址：<https://en.cppreference.com/w/>

Thank you.

把数字世界带入每个人、每个家庭、
每个组织，构建万物互联的智能世界。

Bring digital to every person, home, and
organization for a fully connected,
intelligent world.

**Copyright©2020 Huawei Technologies Co., Ltd.
All Rights Reserved.**

The information in this document may contain predictive statements including, without limitation, statements regarding the future financial and operating results, future product portfolio, new technology, etc. There are a number of factors that could cause actual results and developments to differ materially from those expressed or implied in the predictive statements. Therefore, such information is provided for reference purpose only and constitutes neither an offer nor an acceptance. Huawei may change the information at any time without notice.

